1. Student Details

- **Name:** S Shriram
- **Roll Number:** 24F3004661
- **Email:** 24f3004661@ds.study.iitm.ac.in

2. Project Details

- **Title:** Hospital Management System V2
- **Problem Statement:**
  Hospitals require a centralized system to manage doctor availability, patient appointments, and medical history efficiently, instead of relying on manual or disconnected systems.
- **Approach:**
  The application is built using a client-server architecture. The backend is implemented using Flask and exposes REST APIs for handling business logic and database operations using SQLite. The frontend is developed using Vue.js as a Single Page Application, which communicates with the backend using HTTP requests.

3. AI/LLM Declaration

- **Usage of AI Tools:** Yes
- **Estimated Percentage:** Approximately 25–30%
- **Purpose:**
  AI tools were used for debugging issues, structuring parts of the frontend components, and resolving errors related to Celery and API handling.
- **Manual Work:**
  The overall application design, database schema, API integration, and feature implementation were completed manually. I understand the full working of the project and can modify the code when required.

4. Technologies Used

- **Flask:** Backend framework used to build REST APIs and handle routing.
- **SQLAlchemy:** ORM used to define models and interact with the database.
- **SQLite:** Database used for storing application data.
- **Vue.js:** Frontend framework for building user interfaces.
- **Bootstrap:** Used for styling and responsive design.

- **Redis:** Used for caching and as a message broker.
- **Celery:** Used for handling background and scheduled tasks.

5. Database Schema / Design

Tables Used:

- User – stores login credentials and role (Admin, Doctor, Patient)
- Department – stores specialization details
- Doctor – stores doctor-specific information
- Patient – stores patient details
- Appointment – stores booking details (date, time, status)
- Treatment – stores diagnosis and prescription

Relationships:

- A User is associated with either a Doctor or Patient profile
- Doctor and Patient are connected through the Appointment table
- Each Appointment has a corresponding Treatment record after completion

Design Reasoning:
A common User table is used for authentication, while role-specific details are separated into different tables to keep the schema structured and avoid unnecessary null fields.

6. API Design

The backend is designed as a REST API system.

Key Endpoints:

- /login – user authentication
- /register – patient registration
- /appointments – booking and viewing appointments
- /doctors – fetching doctor details and availability
- /treatment – storing and retrieving treatment details

The frontend communicates with these APIs using HTTP requests and updates the UI dynamically.

7. Architecture and Features

Project Structure:

- backend/ – Flask application, database models, API routes, Celery tasks
- frontend/ – Vue components, routing, UI logic

Data Flow:

1. User interacts with Vue frontend
2. Request sent to Flask API
3. Backend processes request and interacts with database
4. Response returned as JSON
5. Frontend updates UI

Implemented Features:

- Role-based login for Admin, Doctor, and Patient
- Appointment booking and management
- Doctor availability for next 7 days
- Prevention of double booking for same slot
- Appointment status updates (Booked / Completed / Cancelled)
- Treatment history recording and viewing
- Search functionality for doctors and patients
- **Celery Jobs:**
    - Daily reminders for scheduled appointments
    - Monthly report generation
    - CSV export of patient treatment history
- **Caching:**
  Frequently accessed data is cached using Redis to improve performance

8. Video Link

https://drive.google.com/file/d/1UY6L0054IhxsdNiL-xBdknqnflzQ2wUw/view?usp=sharing